LDscnR-multi

A User Manual

October 7, 2026

Abstract

This manual explains how to run LD-aware association scans with `LDscnR-multi`, either for one dataset or for hundreds in a batch. The workflow uses the `LDscnR` R package to group correlated markers before testing, report candidate regions, and compare discovery counts with structure-aware null analyses. Examples use full-genome three-spined (117 individuals, 790,578 markers) and nine-spined stickleback (149 individuals, 1,195,557 markers) panels. Smaller two-chromosome datasets are provided for a quick first run.

Contents

1 Introduction **3**
- 1.1 What problem this pipeline solves . . . 3
- 1.2 Relationship to `LDscnR` and `LDscnR-paper` . . . 3
- 1.3 Design philosophy: receipts, not blind reruns . . . 3

2 Key concepts **4**
- 2.1 Stage 1 clustering: units and `core_snp` . . . 4
- 2.2 `ld_w`: local LD support . . . 4
- 2.3 `size_floor`: the minimum tested cluster size . . . 4
- 2.4 Regions: assembling significant units . . . 4
- 2.5 Permutation nulls and why permutation function matters . . . 5

3 Installation and setup **5**
- 3.1 The `LDscnR` version pin . . . 5

4 How many markers should be included? **5**

5 Dataset folder contract **6**
- 5.1 `input/` — what you provide . . . 6
  - 5.1.1 A real `perm_fun.R` . . . 7
  - 5.1.2 What `genotypes.rds`/`map.rds`/`phenotype.rds` actually look like . . . 7
- 5.2 `cache/` and `output/` — what the pipeline writes . . . 7

6 Running the pipeline **7**

7 Pipeline stages in detail **8**
- 7.1 Stage 1: LD-based clustering . . . 8
- 7.2 LD-decay report . . . 8
- 7.3 Structure-alignment diagnostic . . . 9
- 7.4 Floor profile: a lightweight sensitivity check on `size_floor` . . . 12

7.5 The heavier alternative: `run_floor_sweep()` . . . 14
7.6 Stage A: EMMAX association testing . . . 14
7.7 Stage B: LD-aware outlier detection . . . 15
7.8 Manhattan plots . . . 17

8 Batch summary: `run_all()` 17

9 Configuration reference 19

10 Caching and receipts 21

11 Known limitations 21

A Full output directory tree 21

1 Introduction

`LDscnR-multi` is a script-based workflow, not a second R package. Install `LDscnR` from GitHub, then use this repository's scripts to analyse one dataset or a collection of dataset folders. Each dataset keeps its own inputs and results; a batch run also produces a combined summary.

On the machines used for the examples, a full run took approximately **12 minutes for 790,578 markers** (three-spined stickleback) and **24 minutes for 1,195,557 markers** (nine-spined stickleback). These are measured examples, not promised runtimes: marker density, memory, processor speed, permutation count and plotting choices matter. The association step is comparatively quick because `emmax_fast()` reuses the fitted relationship model while scanning markers or reduced units; Stage-1 LD calculations and the floor profile account for much of the remaining time.

1.1 What problem this pipeline solves

A genome scan for markers associated with a phenotype (or an ecological gradient) faces a structural problem: markers in linkage disequilibrium (LD) with each other are not independent tests. Naively testing every marker treats a single underlying signal — one causal region — as though it were dozens or hundreds of separate discoveries, inflating both the apparent strength of a hit and the multiple-testing burden of the whole scan.

`LDscnR`'s answer is a two-stage design, and `LDscnR-multi` runs both stages, per dataset:

- **Stage A** (`run_stage_A()`): genotypes + a marker map + a phenotype are turned into association p-values, using EMMAX (a mixed model correcting for relatedness/population structure via a genomic relationship matrix, GRM). Two independent *arms* are computed (see Section 7.6):
  - `unit` — one test per LD-cluster (a "unit"), collapsing correlated markers into one representative test;
  - `simes` — one p-value per individual marker, aggregated up to its cluster via Simes' method.
- **Stage B** (`run_stage_B()`): takes Stage A's cluster-level results, or aggregates marker-level p-values from Stage A or an external method such as LFMM. It identifies significant LD-clusters and assembles nearby significant clusters into reported *regions*.

Everything else in this manual — the LD-decay report, the structure-alignment diagnostic, the floor profile, the Manhattan plots — exists to support, calibrate, or visualise these two stages.

1.2 Relationship to `LDscnR` and `LDscnR-paper`

`LDscnR` supplies the clustering, association and outlier functions. `LDscnR-multi` organises them into a repeatable workflow for one or many datasets. Its example data come from the stickleback analyses in the LDscnR study; you do not need the manuscript or its analysis scripts to follow this manual.

1.3 Design philosophy: receipts, not blind reruns

Every stage records its inputs and settings and skips recomputing when nothing has changed (Section 10). This helps with both a single dataset and large batches.

2 Key concepts

This section defines every piece of pipeline-specific vocabulary used later. Skip it if you already know what an LD-complexity-reduction cluster, a `size_floor`, or a floor profile is.

2.1 Stage 1 clustering: units and `core_snp`

Before any association test runs, `LDscnR`'s `ld_complexity_reduction()` groups markers into clusters (called *units* downstream) purely from genotype LD, with no phenotype involved. Two markers land in the same cluster only if the pairwise LD between them, and everything along a chain connecting them, clears a threshold ρ (this repository's default `cr_rho = 0.5`). Each cluster gets one representative marker, its `core_snp`, which never changes as long as the clustering itself is unchanged — this is what makes `core_snp` the right key for comparing clusters across a floor profile (Section 7.4), where a cluster's position index (`unit_id`) is *not* stable.

2.2 `ld_w`: local LD support

`ld_w` (reported at $\rho = 0.95$ as `ld_w_095` throughout this pipeline's output) is a per-marker statistic answering: "how much local LD support does this marker have?" It comes from fitting a decay curve $r^2 \sim b + (c-b)/(1+a \cdot d)$ to how LD falls off with distance d on each chromosome, then asking, for each marker, how far its own local LD extends before dropping below the ρ threshold. A marker sitting inside a large block of mutually correlated markers (e.g. an inversion, or a region of suppressed recombination) has high `ld_w`; a marker in an isolated, low-LD stretch of the genome has low `ld_w`. It is *not* a test statistic and carries no p-value of its own — it is a structural, genotype-only property of the marker's neighbourhood, reported alongside every association result as context (Section 7.2), and plotted directly as an alternative Manhattan-plot axis (Section 7.8).

2.3 `size_floor`: the minimum tested cluster size

Not every Stage-1 cluster is tested. `size_floor` is the minimum number of markers a cluster must contain to be tested at all; smaller clusters are skipped, never assigned a p-value. This reduces the number of tests and makes the minimum support required of a tested cluster explicit. The pipeline requires at least two markers per tested cluster. Unless set explicitly, the floor is derived from marker density: one required member per 250 markers per Mb of assayed sequence, rounded (`size\_floor\_markers\_per\_mb` in `DEFAULTS`). A density, unlike a raw marker count, does not change when only some chromosomes are analysed, so a worked example built from two chromosomes gets the same floor as its full panel. Small clusters can nevertheless contain real signals; raising the floor trades genomic coverage for a smaller, more concentrated test set (Section 7.4).

2.4 Regions: assembling significant units

A significant unit does not stand alone in the reported output. Once Stage B has decided which units clear the significance threshold, it assembles physically nearby significant units into a *region* (one row of `region_table.csv`, Section 7.7). Two assembly modes exist: `stage2_discovered` (the default — a second round of `LDscnR`'s own clustering, at a possibly different LD threshold, run on just the significant units) and `physical` (a simpler gap-merge of significant units' own physical spans). This manual's example output uses the default.

2.5 Permutation nulls and why permutation function matters

Calibration repeats the association analysis with surrogate phenotypes to see how many discoveries could arise without the observed phenotype–genotype relationship. *How* the surrogate is constructed matters: shuffling individuals directly, ignoring population structure, can break the very structure a mixed model is meant to account for. `LDscnR-multi` therefore requires every dataset with an EMMAX phenotype scan to supply its own `perm_fun.R` (Section 5) rather than assuming a generic shuffle is safe.

3 Installation and setup

Install the validated `LDscnR` revision from GitHub, then download this repository. The first command needs internet access; subsequent analyses run from local files. The validated revision is recorded in `R/00_config.R` as `LDSCNR_PIN$sha`.

```
install.packages("remotes")               # once, if needed
remotes::install_github("genomeinsights/LDscnR@2fc1d44a2552")
library(LDscnR)
```

`LDscnR-multi` itself is not installed as a package. From its repository directory, either source its scripts in R:

```
library(LDscnR)
for (f in sort(list.files("R", pattern = "\\.R$", full.names = TRUE))) source(f)
run_dataset("examples/3sp_chr1_chr4")
```

or, from the shell, `run_batch.R` does both steps for you and then runs the pipeline over one or more dataset folders:

```
Rscript run_batch.R examples/3sp_chr1_chr4 examples/9sp_chr19_chr20
Rscript run_batch.R examples/                 # every subfolder with an input/
```

3.1 The `LDscnR` version pin

The pipeline checks the GitHub commit recorded in the installed package, not merely its version number. If it does not match the validated revision, the run stops before processing a dataset. This prevents a large batch from mixing results obtained with different package code. Reinstall the revision shown above when that happens. Developers working from a local checkout may set `LDSCNR_DEV_LOAD_ALL=1` before using `run_batch.R`; that mode loads the checkout and checks its source instead. Ordinary users need neither a local `LDscnR` checkout nor `devtools`.

4 How many markers should be included?

For most applications, we suggest starting with a well-distributed panel of no more than approximately **one million filtered markers**. This is a practical target, not a biological or software limit. The default floor profile ultimately tests only the larger LD clusters, so adding many densely packed markers can increase LD calculations and plotting time without adding many distinct tests. The examples show that even a 1.2-million-marker panel is feasible, but it takes longer and uses more memory than the 790,578-marker panel.

If the starting panel is substantially denser, thin it *before* building the input files. First apply the study's usual missingness, MAF and quality filters. Then use a recorded random seed to sample markers across every chromosome and across short physical bins within chromosomes, limiting very dense bins while retaining several markers per bin. Set bin width and per-bin sampling quotas from the genome length and desired total so that broad coverage is maintained; do not

select markers using their phenotype association or keep only one marker from each LD cluster, which would defeat the cluster-based test. Apply the same selected marker IDs and order to `genotypes.rds`, `map.rds` and any `external_pvalues/` vectors. Record the filter, seed, binning rule and retained marker count.

Thinning *can* change LD estimates, cluster sizes and discoveries. An outlier region represented by too few retained markers may disappear, whereas additional markers in an unthinned panel could have made its LD pattern testable. Therefore, when possible, compare the floor profile and main candidate regions on one representative denser dataset before applying the chosen thinning rule to hundreds of datasets. The one-million target should never be interpreted as proof that denser data cannot reveal additional regions.

5 Dataset folder contract

Every dataset is one self-contained folder. Nothing about it is registered anywhere else; passing its path to `run_dataset()` is the only thing that ties it into the pipeline.

5.1 `input/` — what you provide

File	Required?	Contents
`genotypes.rds`	required	An $n \times m$ numeric dosage matrix (values in $[0, 2]$, `NA` allowed), individuals in rows (`rownames` = individual IDs), markers in columns (`colnames` = marker IDs).
`map.rds`	required	A `data.table`/`data.frame` with columns `marker`, `Chr`, `Pos`. `map$marker` must equal `colnames(genotypes)`, in the same order.
`phenotype.rds`	only for Stage A	A named numeric vector (names = `genotypes`' rownames), or a one-column data frame with those row names.
`perm_fun.R`	required for Stage A	Plain R defining `perm_fun(b, y)`, returning one permuted copy of phenotype `y` for permutation index `b`. There is deliberately no generic default (see Section 2); see the real example below.
`population.rds`	only for the structure-alignment diagnostic	Named vector, individual → population ID.
`structure_group.rds`	optional	Named vector, individual → a coarser sampling group (locality, lineage, ...). Defaults to `population.rds` when absent; this is the grouping a permutation is actually restricted within.
`config.R`	optional	Plain R assigning any `DEFAULTS` name from `R/00_config.R` (Section 9) to override it for this dataset only, e.g. `cr_rho <- 0.35`.

`external_pvalues/` is a second, optional top-level folder: one `.rds` per extra p-value engine, each a *named* numeric vector (names = `map$marker`). The file name, minus `.rds`, becomes the engine name — `external_pvalues/lfmm.rds` is run through Stage B as engine `lfmm`. This is

how `LDscnR-multi` stays engine-agnostic: Stage B does not care whether a p-value came from its own Stage A or from an entirely different method, as long as it is one p-value per marker.

5.1.1 A real `perm_fun.R`

The example's permutation function is shown below. The phenotype (ecotype) is constant within a population, so an individual-level shuffle would create within-population variation that never exists in the real data; this shuffle instead reassigns *which* population gets which value, restricted to populations sharing the same locality:

```
population <- readRDS(file.path(.dataset_dir, "input", "population.rds"))
structure_group <- readRDS(file.path(.dataset_dir, "input", "structure_group.rds"))

perm_fun <- function(b, y) {
  pop <- population[names(y)]
  grp <- structure_group[names(y)]
  pops <- unique(pop)
  pop_group <- grp[match(pops, pop)]
  y_pop <- as.numeric(y[match(pops, pop)])          # one value per population (constant within)
  y_pop_shuf <- stats::ave(y_pop, pop_group, FUN = sample)   # shuffled within locality, at population resolution
  as.numeric(y_pop_shuf[match(pop, pops)])          # broadcast back to individuals
}
```

5.1.2 What `genotypes.rds/map.rds/phenotype.rds` actually look like

From `full_runs/3sp_full` (117 individuals × 790,578 markers — the corner shown below happens to be identical to the small worked example's own copy, since both start from the same Chr1):

```
> g <- readRDS("input/genotypes.rds"); g[1:4, 1:4]
      Chr1:2480 Chr1:2492 Chr1:2493 Chr1:2494
ind_1  0.007751  0.015384  0.058822  1.000001
ind_2  0.003891  0.003891  0.007751  0.999986
ind_3  1.000000  1.000000  1.000014  0.003891
ind_4  0.000244  0.038510  0.003891  1.000000

> m <- readRDS("input/map.rds"); head(m)
     marker  Chr  Pos
1 Chr1:2480 Chr1 2480
2 Chr1:2492 Chr1 2492
3 Chr1:2493 Chr1 2493
...

> p <- readRDS("input/phenotype.rds"); head(p)
ind_1 ind_2 ind_3 ind_4 ind_5 ind_6
    0     0     0     0     0     0
```

5.2 `cache/` and `output/` — what the pipeline writes

Both are gitignored and fully regenerable: delete either one and rerun to get it back (subject to the receipt system re-deciding what actually needs recomputing, Section 10). `cache/` holds the GDS genotype file, the LD-decay fit, the Stage-1 clustering, and the GRM — everything that depends only on genotypes, never on a phenotype. Note there is deliberately no `cache/edge_lists/`: at full-genome marker density a single chromosome's raw pairwise edge list can run several gigabytes, so it is never persisted to disk — every consumer rebuilds it from the GDS file on the fly, one chromosome at a time, and discards it immediately after use. `output/` is described stage by stage in Section 7, and summarised as a full tree in Appendix A.

6 Running the pipeline

The typical entry point is `run_dataset(dataset_dir)`, which runs every applicable stage, in order, for one dataset. The two-chromosome worked examples under `examples/` are the right thing to point this at first — a full genome panel like this manual's own `full_runs/3sp_full`

(20 chromosomes, 790,578 markers) genuinely takes minutes per stage, not seconds: a real, from-scratch, single-machine timed run of `full_runs/3sp_full` took about 6.3 minutes for Stage 1, 3.8 minutes for the floor profile, under 15 seconds each for the LD-decay report, the structure-alignment diagnostic, Stage A, and each Stage B engine, and about 35 seconds per Manhattan figure — roughly 12 minutes end to end on this 790,578-marker panel; the nine-spine panel (1,195,557 markers) took about 24 minutes. Section 10 covers why a second call on an unchanged dataset is nonetheless cheap:

```
run_dataset("examples/3sp_chr1_chr4")

result <- run_all(c("examples/3sp_chr1_chr4", "examples/9sp_chr19_chr20"))
result$summary            # one row per (dataset, engine)
result$alignment_summary  # one row per dataset with input/population.rds
```

Internally, `run_dataset()` executes the following steps, each gated by its own receipt (so a repeat call after a partial change re-runs only what actually needs it):

1. **Stage 1** (Section 7.1) — genotype-only LD clustering, always runs.
2. **LD-decay report** (Section 7.2) — always runs, genotype-only, no recomputation (just writes what Stage 1 already computed).
3. **Structure-alignment diagnostic** (Section 7.3) — runs only if `input/population.rds` *and* `input/phenotype.rds` both exist.
4. **Floor profile** (Section 7.4) — runs by default whenever `input/phenotype.rds` exists; labels each reported region by its stability at half and twice `size_floor`. It does not change the reported floor.
5. **Stage A** (Section 7.6) — EMMAX association testing, if a phenotype is present.
6. **Stage B, once per engine** (Section 7.7) — Stage A's own arms plus every file under `external_pvalues/`.
7. **Manhattan figures** (Section 7.8) — two multi-panel figures, one per available engine.

7 Pipeline stages in detail

7.1 Stage 1: LD-based clustering

Function: `build_stage1(dataset_dir, cfg)`. **Depends on:** genotypes and map only — no phenotype.

This is the genotype-only foundation everything else builds on: it creates the GDS-format genotype file, fits the genome-wide LD-decay curve (Section 2.2), computes `ld_w_095` for every marker, runs `LDscnR::ld_complexity_reduction()` to group markers into Stage-1 units (`cr_rho`, default 0.5), and builds the genomic relationship matrix (GRM) Stage A's mixed model needs. It writes a single cache file, `cache/stage1.rds`, and nothing under `output/` directly — everything below reads from this cache.

7.2 LD-decay report

Function: `report_ld_decay(dataset_dir, cfg)`. **Writes:** `output/ld_decay/`.

A pure reporting layer over what `build_stage1()` already computed and cached — no recomputation. Three files:

- `decay_summary.csv` — one row per chromosome, described below.

- `decay_print.txt` — LDscnR's own `print.ld_decay()` console output, captured verbatim.
- `decay_curves.pdf` — LDscnR's own `plot.ld_decay()`: one genome-wide summary page, then one page per chromosome (Figure 1).

`decay_summary.csv` — one row per chromosome

20 rows on the full 3sp panel, one per chromosome (first 4 shown):

```
   c         a           b          chr_size n_snp_chr bp_per_snp  Chr
1: 0.9080493 0.002468281 0.05513295 28185489  56654     497.4672   Chr1
2: 0.9203360 0.002705361 0.05513295 23295619  45981     506.6435   Chr2
3: 0.9209214 0.003332347 0.05513295 16797709  34876     481.5797   Chr3
4: 0.8996757 0.001037658 0.05513295 32632357  63953     510.2515   Chr4
   a_pred      c_pred    n_w_used  rho_slide_raw  rho_slide_pred
1: 0.002231920 0.9171189 39        0.9981935      0.9980025
2: 0.002600321 0.9171189 39        0.9983813      0.9983161
3: 0.003379862 0.9171189 39        0.9986172      0.9986366
4: 0.001984574 0.9171189 39        0.9958204      0.9978103
   ... (plus slide_bp_rho_<X> / slide_snp_rho_<X> for X in rho_targets)
```

Column	Meaning
`Chr`	Chromosome.
`chr_size`	Chromosome span in bp (`max(Pos)` on this chromosome).
`n_snp_chr`	Number of markers on this chromosome.
`bp_per_snp`	`chr_size / (n_snp_chr - 1)` — average marker spacing, used to convert every SNP-count column below to/from bp.
`a`, `c`, `b`	This chromosome's *own* fitted decay-curve parameters for $r^2 \sim b + (c-b)/(1+a\cdot d)$: `a` is the decay rate (larger = faster decay), `c` the short-range asymptote, `b` the genome-wide background LD floor (identical every row).
`a_pred`, `c_pred`	The genome-wide, chromosome-size-predicted values (a robust regression of log(*a*) on log(chr_size) across every chromosome). `ld_w_095` and the "size-predicted" curve in `decay_curves.pdf` use these, not this chromosome's own noisier `a`/`c`. Above, `a_pred` genuinely varies chromosome to chromosome (0.00198–0.00451 across all 20) because a real regression across 20 chromosome sizes was fit. *Note:* on the small, two-chromosome `examples/` datasets this regression cannot be fit at all (fewer than 3 distinct chromosome sizes) and degenerates to the plain median of `a` instead — a genuine, expected difference from a full panel, not a bug, and specific to those small worked examples.
`n_w_used`	How many of this chromosome's sliding windows cleared the "structured" contrast-over-background gate and were used to fit `a`/`c`.
`rho_slide_raw`, `rho_slide_pred`	The LD retention ρ the *current* `slide` setting actually covers, using this chromosome's own `a` / the size-predicted `a_pred`.
`slide_bp_rho_<X>`, `slide_snp_rho_<X>`	For each target in `rho_targets` (default 0.90/0.95/0.99): the physical window (bp) / marker-count window (SNPs) needed to retain that much LD, using `a_pred`.

7.3 Structure-alignment diagnostic

Function: `check_structure_alignment(dataset_dir, cfg)`. **Runs when:** `input/population.rds` exists. **Writes:** `output/alignment/`.

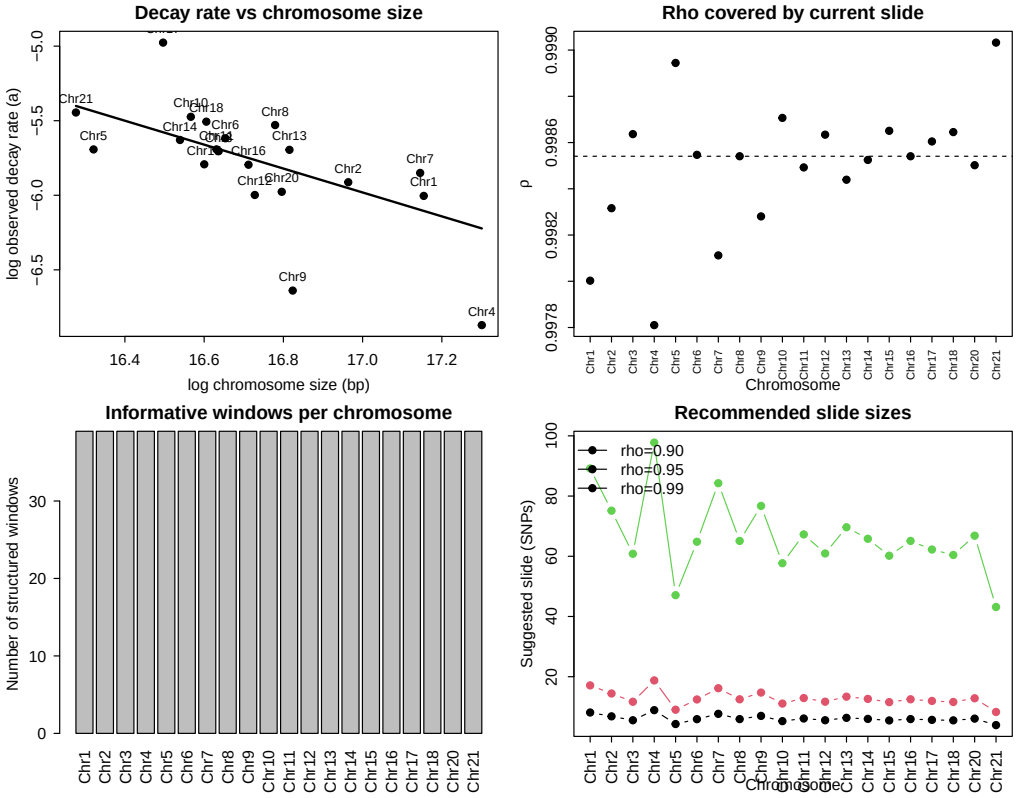


Figure 1: `decay_curves.pdf`, page 1 (genome-wide summary), for `full_runs/3sp_full`: decay rate vs. chromosome size (a real regression across all 20 chromosomes, not the flat line the two-chromosome worked example would show), LD retained by the current `slide` setting, and informative windows per chromosome. Each chromosome then gets its own page (not shown) with its window-wise fit against the pooled data.

Before any association test runs, this asks how much of the tested phenotype is already explained by population structure — specifically, by (a) `structure_group.rds` (the sampling groups a permutation null would shuffle within) and (b) the leading axes of the population-level GRM. **This is a warning diagnostic, not a gate**: a low alignment does not invalidate a result, but a high one means the phenotype and ancestry provide nearly interchangeable explanations for whatever signal Stage A finds, and that is worth knowing before interpreting a hit.

`observed.csv` — one row

```
n_individuals n_populations n_structure_groups grm_axes_r2_population
          117            35                 4              0.1033205
grm_axes_adjusted_r2_population structure_r2_population
                   -0.05127941               0.0541958
structure_r2_individual_weighted
                       0.1113596
```

Column	Meaning
`n_individuals`, `n_populations`, `n_structure_groups`	Sample sizes: individuals, distinct `population.rds` values, distinct `structure_group.rds` values.
`grm_axes_r2_population`	R^2 of population-mean phenotype regressed on the top eigenvectors of the population-level GRM.
`grm_axes_adjusted_r2_population`	The same, adjusted R^2 (penalised for the number of axes used).
`structure_r2_population`	R^2 of population-mean phenotype regressed on `structure_group`, weighted one point per population.
`structure_r2_individual_weighted`	The same, weighted one point per individual (populations with more individuals count more).

`null_draws.csv` — one row per (scheme, draw)

Every draw's own version of `observed.csv`'s three measures, recomputed on a null-shuffled/null-surrogate phenotype. Two null schemes are used: `"within-group permutation"` (shuffle within `structure_group`) and `"GRM-matched continuous"` (a continuous surrogate matched to the GRM's own covariance structure).

```
                   scheme  draw grm_axes_r2_population structure_r2_population
1 within-group permutation     1              0.09180002               0.0541958
2 within-group permutation     2              0.11839943               0.0541958
3 within-group permutation     3              0.15230749               0.0541958
   structure_r2_individual_weighted
1                        0.03626990
2                        0.22767224
3                        0.12800670
```

`null_summary.csv` — one row per (scheme, metric)

`observed.csv`'s three measures compared against their own null distribution.

```
                   scheme                          metric  observed null_mean
within-group permutation          grm_axes_r2_population 0.1033205 0.1300955
  GRM-matched continuous          grm_axes_r2_population 0.1033205 0.3044517
within-group permutation         structure_r2_population 0.0541958 0.0541958
  null_median      null_sd  null_q025 null_q975 observed_within_null95
   0.1197442 5.714492e-02 0.04991410 0.2674327                    TRUE
   0.2924182 1.443470e-01 0.07612137 0.6073546                    TRUE
   0.0541958 1.039326e-16 0.05419580 0.0541958                    TRUE
   proportion_null_ge_observed
```

```
                    0.630
                    0.931
                    1.000
```

Column	Meaning
`scheme`, `metric`	Which null, and which of the three measures.
`observed`	The observed value (from `observed.csv`).
`null_mean`, `null_median`, `null_sd`	That null distribution's own summary.
`null_q025`, `null_q975`	The null's 2.5th/97.5th percentile.
`observed_within_null95`	Whether `observed` falls inside [`null_q025`, `null_q975`] — if `TRUE`, the observed alignment is not distinguishable from this null.
`proportion_null_ge_observed`	A one-sided p-value-like quantity: the fraction of null draws at least as large as `observed`. Low values mean the observed alignment is unusually strong relative to this null.

7.4 Floor profile: a lightweight sensitivity check on `size_floor`

Function: `run_floor_profile(dataset_dir, factors = c(0.5, 2))`. **Runs when:** `input/phenotype.rd` exists, by default (`floor_profile = TRUE` in `run_dataset()`). **Writes:** `output/floor_profile/`.

Why this exists

Section 2.3 introduced `size_floor`, the minimum Stage-1 cluster size that is tested. Any single floor is a practical choice, so the floor profile asks how robust each reported region is to that choice. Every region reported at `size_floor` is checked at *half* and at *twice* that floor (4 and 16 for a floor of 8) and labelled `"3/3"` (found at both), `"2/3"` (one) or `"1/3"` (neither). The low comparison floor may be 1 (singletons included); it is never the reported floor.

In the LDscnR paper's simulations (600 runs with known causal variants), regions reported at the density floor had precision of about 0.2 when found at that floor only, 0.5 when also found at one comparison floor and 0.8 when found at both, for consensus, Simes and LFMM Simes alike. Dropping `"1/3"` regions cost almost no recall. Recovery at the *lower* floor, where more units are tested and the BH threshold is harder to clear, was the more informative of the two checks.

Only the reported floor gets the full, permuted treatment

- The **reported** floor runs Stage A and Stage B exactly as `run_dataset()` always has — permutations, the outlier permutation test, region assembly and, if `annotation` is given, `ld_region_rotation()`.
- The two **comparison** floors are observed only (`run_stage_B(observed_only = TRUE)`), which never loads a cached permutation file. The consensus arm is fitted once more, at the low floor (`run_stage_A(permute = FALSE)`); the high floor's clusters are a subset of the low floor's and a unit's EMMAX p-value does not depend on which other units are tested, so the high floor reuses those p-values and only BH correction and Stage-2 assembly are repeated. The "simes" arm reuses the floor-independent marker scan at every floor.

How a reported region's "stability" is judged

A unit's own `unit_id` is just a position index into that floor's own size-filtered cluster list, and is *not* comparable across floors — two unrelated clusters can both be "`unit_id` 5" at two different floors. Each cluster's `core_snp` (Section 2) is stable across floors, and is what the floor profile actually keys on to ask "is this reported region's cluster *also* a significant unit at this comparison floor?". Physical span overlap is used only as a secondary sanity check on a recovered hit.

Output files

`grid.csv` — one row per floor, on the full 3sp panel (790,578 markers, 20 chromosomes; density floor 8):

```
       role floor achieved_reduction n_units_tested marker_coverage
1:      low     4          0.9832641          13231      0.09144828
2: reported     8          0.9982962           1347      0.02136412
3:     high    16          0.9997103            229      0.00763998
```

`summary.csv` — one compact row per method (`unit`/`simes`):

```
 method floor_low floor floor_high reduction_low reduction reduction_high
   unit         4     8         16     0.9832641 0.9982962      0.9997103
  simes         4     8         16     0.9832641 0.9982962      0.9997103
 n_units_tested n_significant n_regions n_tier_1 n_tier_2 n_tier_3
           1347           100        38       25        9        4
           1347            87        37       25        7        5
```

`canonical_regions_unit.csv` / `canonical_regions_simes.csv` — one row per *reported-floor* region (a region found only at a comparison floor never appears here — this table is about how robust the *reported* findings are, not a census of every floor's own discoveries). 38 rows for the `unit` arm on the full 3sp panel, split 4×"3/3", 9×"2/3" and 25×"1/3":

```
              region_id    Chr     from       to tick_004 overlap_check_004
  Chr1:21492308-21938477   Chr1 21492308 21938477     TRUE              TRUE
  Chr4:22225174-22511008   Chr4 22225174 22511008     TRUE              TRUE
  Chr4:14442422-14562613   Chr4 14442422 14562613    FALSE                NA
tick_016 overlap_check_016 n_ticks stability_tier   span
    TRUE              TRUE       3            3/3 446169
   FALSE                NA       2            2/3 285834
   FALSE                NA       1            1/3 120191
```

Column	Meaning
`region_id`, `Chr`, `from`, `to`, `span`	The reported region's own identity and physical extent.
`tick_<floor>`	`TRUE` if this region's cluster(s) are *also* a significant unit at that comparison floor. A cluster whose `n_loci` does not clear a comparison floor is simply never tested there, which correctly reads as not-recovered — this is not a bug, it is what floor selection means.
`overlap_check_<floor>`	Secondary sanity check, only meaningful when the tick above is `TRUE`: does that comparison floor's own region (containing the recovered cluster) physically overlap this reported region's span? Almost always `TRUE`; a `FALSE` here is a genuine anomaly worth a manual look.
`n_ticks`, `stability_tier`	How many of the 3 floors (reported + 2 comparison) found this region: `"3/3"`, `"2/3"`, or `"1/3"` (never `"0/3"`, since every row starts from a reported, already-found region).

The tier is a robustness label, not a significance measure — never a p-value, an FDR estimate or an independent replication. Every floor analyses the same data, only the reported floor gets the structure-aware permutation null, and a region that persists across floors can still be generated by population structure. The simulation precisions above make the label a useful ranking aid, not a calibrated probability of truth in a new dataset.

7.5 The heavier alternative: `run_floor_sweep()`

Function: `run_floor_sweep(dataset_dir, factors = c(0.5, 1, 2))`. Not run by default inside `run_dataset()`.

Where the floor profile above deliberately keeps two of its three floors cheap (observed-only), `run_floor_sweep()` reruns *full* permutations at every floor it tests (half/default/double the resolved `size_floor`). This is a genuinely more expensive, more thorough robustness check, appropriate for a one-off analysis of a single dataset, not for a batch of hundreds. Call it separately when you want that extra assurance.

`floor_sweep_summary.csv` — one row per (floor, statistic):

```
 floor statistic n_units_tested n_significant n_regions
     8      unit            337            62        13
     8     simes            337            51         9
```

Column	Meaning
`floor`	The resolved integer `size_floor` this row used.
`statistic`	`"unit"` or `"simes"`.
`n_units_tested`	How many units cleared this floor and were tested.
`n_significant`	How many of those were significant at `alpha`.
`n_regions`	How many regions the significant units assembled into.

7.6 Stage A: EMMAX association testing

Function: `run_stage_A(dataset_dir, cfg)`.

Writes: the `emmax_unit` and `emmax_simes` subfolders of `output/pvalues/`.

Both arms use the same EMMAX mixed model (genotypes, phenotype, the GRM from Stage 1) but differ in what gets tested:

- **`unit`**: one test per Stage-1 cluster that clears `size_floor`, using a consensus representation of the cluster's member markers (`unit_repr = "consensus_dosage"` by default). Writes `p_obs.rds`/`p_perm.rds` (one value per tested unit), plus `p_obs_marker.rds` — a genuine, separate per-marker EMMAX scan computed purely so this arm's Manhattan plot can show real per-SNP variation instead of one flat value repeated across every marker in a unit.
- **`simes`**: every marker is tested directly, then aggregated up to its cluster's p-value via Simes' method — but only for clusters that clear `size_floor`. Writes `p_obs.rds` for *every* marker (needed for full-resolution plotting regardless of what is actually tested) and `p_perm_compact.rds`, a permutation matrix restricted to markers whose cluster clears the floor. At full-genome scale this restriction is the single biggest cost saving in the whole pipeline: on the full 3sp panel, only 16,890 of 790,578 markers (2.1%) sit in a cluster clearing `size_floor = 8`, so rescanning the other 97.9% on every permutation would be pure waste.

These outputs are R objects (`.rds`), not tables to preview here — Section 7.7 shows what they turn into once Stage B reads them.

7.7 Stage B: LD-aware outlier detection

Function: `run_stage_B(dataset_dir, engine, cfg)`, once per available engine. **Writes:** `output/stageB_<engine>/`.

This is where a vector of per-marker p-values becomes a decision: which clusters are significant, and which physically contiguous regions do they assemble into. Engine-agnostic by design — Stage A's own `emmax_unit` and `emmax_simes` arms, and any `external_pvalues/*.rds` file, go through the identical call. `full_runs/3sp_full` demonstrates this directly: it has an `external_pvalues/lfmm.rds`, so its Stage B runs three times (`emmax_unit`, `emmax_simes`, `lfmm`) — compare the `lfmm` region table below (57 regions) against `emmax_unit`'s (44 regions): same columns, same meaning, from a completely different upstream method, over the same 790,578-marker panel.

`snp_results.csv` — one row per marker in `map`

The first five rows of *any* panel are mostly `NA`, since most markers are not tested; a filtered view (`tested == TRUE & significant == TRUE`) is more informative:

```
       marker  Chr      Pos statistic marker_p marker_q      unit_p     unit_q
Chr1:19374296 Chr1 19374296      unit       NA       NA 0.002316167 0.04711038
Chr1:19374302 Chr1 19374302      unit       NA       NA 0.002316167 0.04711038
 p_display  q_display unit_id              region_id significant tested   ld_w_095
0.10035796 0.87447990      90 Chr1:19374296-19374576        TRUE   TRUE 0.03257659
0.04846713 0.81744030      90 Chr1:19374296-19374576        TRUE   TRUE 0.02403367
```

Column	Meaning
`marker`	Marker ID, from `map$marker`.
`Chr`, `Pos`	Chromosome and position, from `map`.
`statistic`	`"unit"` or `"simes"` (or whatever `statistic` an external engine was run with) — which arm produced this row.
`marker_p`, `marker_q`	The marker's own raw p-value / BH q-value. Only meaningful when `p_obs` is marker-aligned (`statistic = "simes"`, or any external engine); `NA` for `statistic = "unit"`, where no per-marker p-value exists at all.
`unit_p`, `unit_q`	The marker's tested unit's p-value/q-value, broadcast to every member marker. This is what `significant` is decided from, for *both* statistics.
`p_display`, `q_display`	What the Manhattan plot (Section 7.8) actually plots. Identical to `marker_p`/`marker_q` when those exist; for `statistic = "unit"`, the separate per-marker EMMAX scan mentioned in Section 7.6 (`p_obs_marker.rds`) — distinct from the flat broadcast `unit_p`, so the "unit" engine's plot still shows real per-SNP variation instead of one flat value per cluster.
`unit_id`	Which Stage-1 cluster (that cleared `size_floor`) this marker belongs to, or `NA` if the marker's cluster did not clear the floor (never tested).
`region_id`	Which assembled region this marker's unit belongs to, or `NA` if its unit was not significant / not assembled into any region. String form `"<Chr>:<from>-<to>"`.

Column	Meaning
`significant`	Whether this marker's unit cleared `alpha` after BH correction. `NA` when `tested` is `FALSE`.
`tested`	Whether this marker's unit cleared `size_floor` and was actually tested (equivalent to `!is.na(unit_id)`).
`ld_w_095`	Local-LD support at $\rho = 0.95$ (Section 2.2), carried from `map`. Identical across every engine's `snp_results.csv` for this dataset — it is not a property of any one test.

Note on membership. Assignment to `unit_id`/`region_id` is by exact cluster/group membership, never by whether a marker's coordinate merely falls inside a region's bounding span. Stage-1 clusters, and the groups region assembly merges them into, are not necessarily contiguous, so a wide region span can physically contain markers that actually belong to a different, interleaved cluster.

`region_table.csv` — one row per assembled region

```
  Chr     from       to n_units n_markers occupancy
 Chr1 19374296 19374576       1        10 0.5263158
 Chr1 21492308 21938477      47      1375 0.5517657
 Chr1 26932748 26932776       1         7 1.0000000
```

Column	Meaning
`Chr`, `from`, `to`	The region's genomic span (bp).
`n_units`	How many of this engine's significant units were assembled into this region.
`n_markers`	How many markers belong to this region's own assembled units (or group).
`occupancy`	`n_markers` divided by how many markers on `map` physically fall within `[from, to]`. 1.0 means every marker in the region's own physical span actually belongs to it; lower values mean other, unrelated markers are interleaved within the same bounding box — a lower-occupancy region is a looser, less contiguous signal.

`report.txt`

`LDscnR`'s own console summary methods (`print.ld_outlier_test()`, `print.ld_outlier_perm()`, `print.ld_region_rotation()`), captured verbatim — no separate summary logic of this repository's own:

```
<ld_outlier_test> unit / stage2_discovered | floor = 7, alpha = 0.05 | 104/2095 units -> 44 regions

<ld_outlier_perm> observed 104 | surrogate mean 2.71 | p = 0.0020 | null/obs ratio 2.6%
```

The nine-spine panel's own `report.txt` (`full_runs/9sp_full`) is a useful contrast: 110 significant units out of 3,498 tested, but the permutation test itself is *not* significant overall (surrogate mean 86.62, close to the 110 observed) — a reminder that many individually BH-significant units do not automatically mean the panel-wide consensus test is significant; the two are asking different questions.

```
<ld_outlier_test> unit / stage2_discovered | floor = 11, alpha = 0.05 | 110/3498 units -> 73 regions

<ld_outlier_perm> observed 110 | surrogate mean 86.62 | p = 0.2727 | null/obs ratio 78.7%
```

7.8 Manhattan plots

Function: `ldm_manhattan(dataset_dir, engines, value)`.

Writes: Manhattan plots in `output/figures/`, showing either association q-values or the local LD measure `ld_w_095`. The latter lets a reported region be compared with the local LD pattern; it is not a second association test.

One panel is drawn per available engine, stacked on a single shared, continuous genome-wide x-axis (chromosomes concatenated, not faceted, with alternating light-grey bands marking where each chromosome starts and ends). Non-significant markers are always a grey background layer; significant markers are coloured by their assembled region, using a locally contrastive colour assignment so that two physically nearby regions are never given confusingly similar hues. The most significant regions per panel are labelled directly (not via a legend); whenever a floor profile (Section 7.4) has been run for that engine's statistic, the label also carries its stability tier, e.g. `"R5 (3/3)"`.

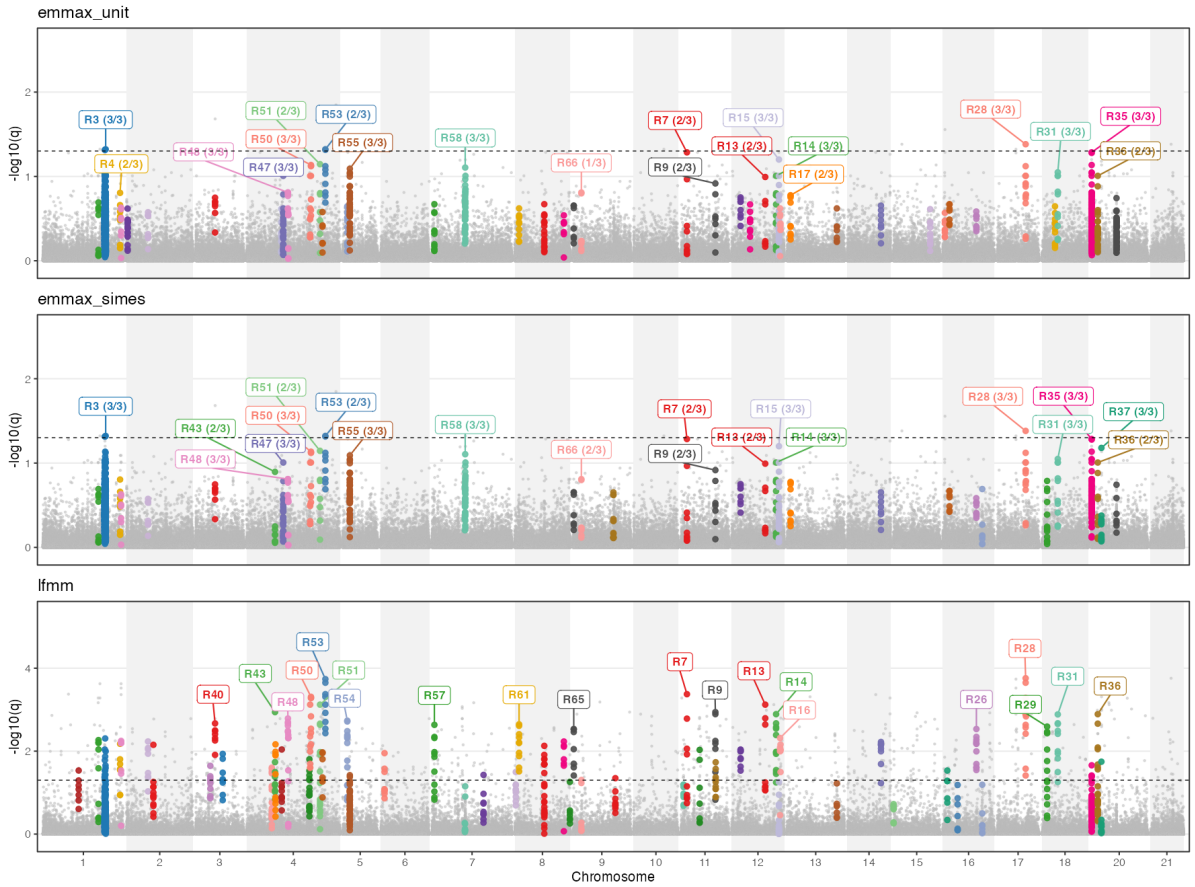


Figure 2: `output/figures/manhattan.png` for `full_runs/3sp_full` (full genome, 20 chromosomes) — q-value on the y-axis, one panel per engine (`emmax_unit`, `emmax_simes`, `lfmm`), stability tiers shown on the two EMMAX panels.

8 Batch summary: `run_all()`

Function: `run_all(dataset_dirs)`. Loops `run_dataset()` over every folder, isolates one dataset's failure from the rest (recorded as `status = "error"`, not an abort), and writes two roll-up CSVs at the top level.

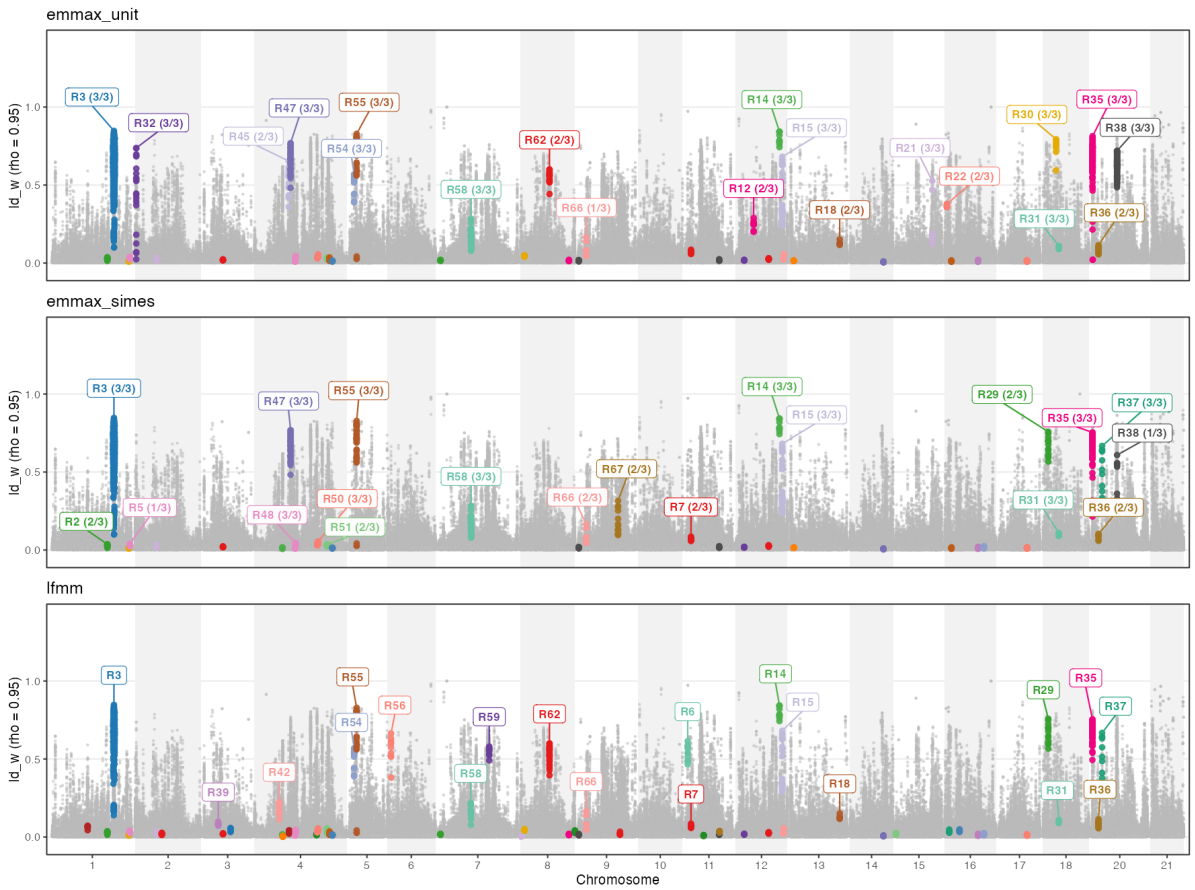


Figure 3: The same dataset, same panels, `ld_w_095` on the y-axis instead of q-value.

summary.csv — one row per (dataset, engine)

Column	Meaning
dataset	basename(dataset_dir).
engine	Which p-value engine this row is for (emmax_unit, emmax_simes, or an external engine's name). NA if the dataset had no engine to run, or errored before reaching one.
n_markers	Total markers in this dataset's map.
n_units_tested, n_significant, n_regions	Same meaning as floor_sweep_summary.csv's columns, for this engine's actual (non-swept) run.
runtime_s	Wall-clock seconds for this dataset's whole run_dataset() call — shared across every engine row for that dataset, not per-engine timing.
status	"ok", "no_engine" (ran, but had no p-value source), or "error".
error	The error message, when status = "error"; NA otherwise.
align_grm_axes_r2, align_grm_axes_p, align_structure_r2, align_structure_p	The two headline structure-alignment numbers, repeated onto every engine row for this dataset so a screen of many datasets can sort/filter on association results and confounding risk at once. All NA when the dataset has no input/population.rds.

The four align_* columns use only the "within-group permutation" null scheme. align_grm_axes_r2 and align_structure_r2 are that metric's observed value; align_grm_axes_p and align_structure_p are its proportion_null_ge_observed — low values flag a phenotype whose signal is hard to distinguish from ancestry/sampling structure alone.

alignment_summary.csv is the detailed companion, one row per dataset with input/population.rds: dataset, then every observed.csv column, then three columns per (scheme, metric) pair from null_summary.csv, named as follows (scheme with any non-alphanumeric characters collapsed to _):

```
<scheme>__<metric>__null_mean
<scheme>__<metric>__observed_within_null95
<scheme>__<metric>__proportion_null_ge_observed
```

9 Configuration reference

Precedence: R/00_config.R's DEFAULTS < a dataset's own input/config.R < an explicit cfg = argument at the call site.

Name	Default	Meaning
cr_rho	0.5	LD threshold for Stage-1 clustering (ld_complexity_reduction()).

Name	Default	Meaning
`decay_args$slide`	450	Window size (in SNPs) for the raw pairwise edge list every LD computation builds per chromosome. Calibrated down from `compute_LD_decay()`'s own default of 1000 specifically to keep the on-the-fly rebuild (see `cache/`, Section 5) small at full-genome marker density; verified not to move `ld_w` meaningfully relative to a larger window.
`decay_args` (other fields)	see `R/00_config.R`	The remaining LD-decay fitting parameters (MAF filter, background-LD quantile, subsampling sizes, robust-fit probability, number of decay windows, ...) — see `decay_summary.csv`'s glossary (Section 7.2) for what they feed into.
`statistics`	`c("unit","simes")`	Which Stage-A arm(s) to compute.
`unit_repr`	`"consensus_dosage"`	How a Stage-1 cluster's member markers are collapsed into one representation for the "unit" arm.
`grm_method`	`"GCTA"`	`SNPRelate::snpgdsGRM()` method used to build the genomic relationship matrix.
`b_unit`	1000	Permutations for the "unit" arm.
`b_simes`	200	Permutations for the "simes" arm (more expensive per permutation, since every eligible marker is rescanned).
`size_floor`	`NULL` (derived)	Minimum tested cluster size — see Section 2.3. `NULL` derives it from marker density; an explicit number pins the floor. Always clamped to ≥ 2.
`size_floor_markers_per_mb`	250	Marker density (markers per Mb) per unit of derived floor: floor = round(density / 250).
`alpha`	0.05	Significance threshold after BH correction.
`assembly`	`"stage2_discovered"`	How significant units are merged into regions: a second clustering pass on just the significant units (`"stage2_discovered"`), or a simple gap-merge of their physical spans (`"physical"`).
`score_threshold`	0.80	LD threshold used by `"stage2_discovered"` region assembly.
`distance_threshold`	10^5	Maximum physical distance (bp) `"stage2_discovered"` assembly will merge across.
`gap`	3×10^5	Maximum gap (bp) `"physical"` assembly will merge across.
`n_rotations`	10000	Rotations for `ld_region_rotation()` (only used when an `annotation` is supplied).
`rotation_scheme`	`"within"`	Rotation scheme for the same.
`seed`	1	Random seed.

Name	Default	Meaning
`cores`	1	Parallel workers for permutation-heavy steps. Increasing this forks EMMAX-permutation workers, each holding its own copy of the working data — watch peak memory at high marker counts (see this repository's `full_runs/*/input/config.R` for a worked example of dropping from 8 to 4 after observing memory pressure at 8).

10 Caching and receipts

Every stage — Stage 1, Stage A, each Stage B analysis, and the structure-alignment diagnostic — hashes its own inputs and parameters into a `_receipt.rds` file next to its output, and skips recomputing when nothing has changed. These are **content** hashes, not timestamps, so a `git checkout` or a copied file cannot produce a false "unchanged". Calling `run_dataset()`/`run_all()` again on an unchanged dataset is a cheap no-op walk over receipts. Pass `force = TRUE` to any stage function to rebuild regardless.

A receipt is also treated as stale if a recorded output is missing or the validated `LDscnR` package revision changes. Use `force = TRUE` only when you intentionally want to rebuild results despite unchanged inputs and settings.

11 Known limitations

- **Plotting at genome scale.** `ldm_manhattan()` plots every marker by default; Figures 2–3 show this works fine at a real, 790,578-marker full-genome scale (about 35 seconds per figure, Section 6), but a point-thinning or rasterisation pass would still be worth adding for a substantially larger panel than this one.
- **One dataset folder per (genotype panel, phenotype) pair.** Screening many phenotypes against the same genotype panel currently means one folder per phenotype, each independently rebuilding Stage 1 (genotype-only, and therefore identical across them) and duplicating `genotypes.rds`/`map.rds` on disk. This is fine when "dataset" genuinely means a different genomic panel, but a real cost when it means many phenotypes on one panel — splitting genotype-panel-level shared state from phenotype-level per-test state would be a real architecture change, not a bolt-on fix.
- **Scope.** Stage B is engine-agnostic on *p-values*, not on detection method — there is no plugin interface for non-`LDscnR` outlier callers (OutFLANK, BayeScan, pcadapt, . . .) that a wider comparative project might also want to use.

A Full output directory tree

```
<dataset_dir>/
  input/
    genotypes.rds, map.rds, phenotype.rds, perm_fun.R,
    population.rds, structure_group.rds, config.R
  external_pvalues/
    <engine>.rds                          # one per extra p-value engine
  cache/                                  # gitignored, regenerable
    genotypes.gds, stage1.rds
  output/                                 # gitignored, regenerable
```

```
    ld_decay/
      decay_summary.csv, decay_print.txt, decay_curves.pdf
    alignment/
      observed.csv, null_draws.csv, null_summary.csv
    pvalues/emmax_unit/
      p_obs.rds, p_perm.rds, p_obs_marker.rds
    pvalues/emmax_simes/
      p_obs.rds, p_perm_compact.rds
    floor_profile/
      grid.csv, summary.csv,
      canonical_regions_unit.csv, canonical_regions_simes.csv
    stageB_<engine>/
      outlier_test.rds, outlier_perm.rds, region_rotation.rds,
      snp_results.csv, region_table.csv, report.txt
    figures/
      manhattan.png, manhattan.pdf,
      manhattan_ld_w.png, manhattan_ld_w.pdf
    floor_sweep_summary.csv               # only after run_floor_sweep()
    figures/manhattan_floor_sweep.{png,pdf}

<repo root>/
  summary.csv, alignment_summary.csv      # only after run_all()
```